

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Designing the Server-Side REST API Flow and Initial API Contract for Your Own Project

Server-Side and REST API Concepts

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 7
Output	API flow diagram, request-lifecycle trace, initial API contract, endpoint and validation plan
Maximum Score	80 points

Student / Project Information

Block: <u>A-1</u>	Date: <u>September 20, 2026</u>
System Title: Seaweed (Guso) Quality Classification Using Image Processing	Instructor: Jean Y. Villasin, MIT
Members: Zydane Angelo Adanza, Timothy Josua Tavera, Jherick Peñalosa	

CARAGA STATE UNIVERSITY – CABADBARAN CAMPUS IT 110 | LAB ACTIVITY 7

1. Activity Alignment and Purpose

Topic Learning Outcome: Explain the role of the REST API backend and design the request–response contract that links Flutter Web to the PostgreSQL data layer.

Course Outcome Alignment: Supports semester-project server-side planning by connecting the approved Flutter Web client design to an instructor-approved REST API backend and the PostgreSQL data layer developed in IT 108.

Syllabus Assessment: API Flow Diagram; Initial API Contract; online request–response tracing exercise.

Purpose of the Activity

Activity 6 established the Flutter Web UI flow and interface behavior. In Activity 7, your team will define what happens after the Flutter client sends a request: which endpoint receives it, how the server validates and authorizes it, which service/business rule executes, how PostgreSQL data is accessed, and what structured response returns to the client. The API flow diagram and initial API contract will become the technical bridge between your Flutter Web screens and the data layer implemented in IT 108.

Important distinction: This activity designs the server-side flow and API contract. It does not duplicate database modeling, normalization, constraints, indexes, transactions, stored procedures, or other database-design work completed in IT 108.

2. Intended Learning Objectives

- Explain the responsibility of the server-side application and REST API layer in the complete web-system architecture.
- Plan REST endpoints, routes, controllers/handlers, service-layer responsibilities, and conceptual data-access responsibilities for the team's actual system.
- Separate transport concerns, server-side validation, authorization, business logic, and data access into clear responsibilities.
- Define JSON request and response bodies, HTTP methods, status codes, required headers, and structured error responses.
- Trace one important request from the Flutter Web interface through the REST API to PostgreSQL and back to the user interface.
- Produce an initial API contract that is traceable to the approved requirements, use cases, UI flow, and PostgreSQL data domains.

3. Activity Specifications

Mode	Existing semester-project group; all members must participate and understand the API flow.
Suggested Time	One 2-hour laboratory session, with outside time allowed for refinement before submission.
Primary Output	One API Flow Diagram for the team's own system.
Supporting Output	Request-lifecycle trace, endpoint inventory, initial API contract, server-validation/error plan, and conceptual API-to-PostgreSQL mapping.
Recommended Tools	diagrams.net/Draw.io, Figma, Lucidchart, PowerPoint/Canva for diagrams; spreadsheet/Word/Docs for contract tables; Postman/Bruno/Insomnia may be used later for testing.
Scope Rule	Plan only endpoints required by the approved semester-project scope. Do not invent unused features or duplicate the IT 108 database design.

4. Pre-Laboratory Requirements

- Approved system title, problem statement, scope, functional/non-functional requirements, and user roles from Activity 5.
- System-specific Use Case Diagram from Activity 5 and Flutter UI-flow/wireframes from Activity 6. • Architecture blueprint and prescribed Flutter Web → REST API → PostgreSQL stack from the earlier architecture/technology-stack activities.
- Instructor-approved backend framework/runtime for the REST API.
- The current PostgreSQL table/entity names or data domains from IT 108 for reference only; do not recreate the ERD in this activity.
- At least one critical user workflow that can be traced from the Flutter Web interface to a database operation and back.

5. Part A – Server-Side Responsibility Map

Before drawing the API flow, assign each concern to a clear server-side responsibility. Use the actual technologies and project terms your team plans to implement.

Server-Side Concern	Your Planned Component / Decision	Notes / Evidence
Request entry / routing	FastAPI APIRouter	POST /api/classifications; routes incoming HTTP requests
Controller / handler	FastAPI Endpoint Functions	Parses request body; extracts image file; calls internal service functions; returns JSON response.
Authentication	FastAPI Depends (JWT Bearer)	Extracts and verifies the JWT token from the Authorization: Bearer <token> header.
Authorization	Role-based verification	Verifies if the authenticated user is a Farmer or Seller allowed to upload images.
Server-side validation	FastAPI validation / Pydantic	Validates file type, size, and corruption; ensures request body matches expected schema
Service / business logic	Image Processing & MobileNetV3	Approval, recommendation, calculations, workflow rules
Data access	SQLAlchemy / asyncpg	Repository/query layer; parameterized query or ORM access
PostgreSQL data domain	Users, Guso_Images, Classification_Results, History	Reference table/entity names from IT 108 only
External service, if any	None	Maps/GIS, ML service, email/SMS, weather, storage, etc.
Response / error handling	FastAPI Exception Handlers	Status code, response JSON, safe error message, logging

6. Part B – Request-Lifecycle Tracing Exercise

Select one critical workflow from your own system. Examples: log in, submit a report, create a record, approve a request, retrieve a map/list, update a profile, or request a recommendation.

Selected workflow: Classifying a Guso image. Flutter screen / user action: Flutter screen / user action - The user selects a Guso image on the Classification Screen (scr_classify) and taps "Classify".

Expected final result shown to the user: The Guso quality grade (for ex. "Grade A"), confidence score, and a success message are displayed on the screen, and the record is saved to history.

Step	Actor / Component	Action or Data	Method / Route / Mechanism	Validation / Security	Expected Result
1	Farmer/Seller	Selects image and taps "Classify".	UI Interaction	Client validates file is < 5MB and JPG/PNG	Loading spinner appears; file is ready.
2	Flutter Web (Riverpod)	Constructs request with image and JWT token	POST /api/classifications (Multipart/form-data)	Attaches Authorization: Bearer <token> header.	HTTPS request sent to backend.
3	FastAPI (Routing)	Receives and routes request to handler.	FastAPI Router match for /api/classifications	Checks if route exists.	Request forwarded to handler.
4	FastAPI (Security)	Verifies JWT token and extracts user ID.	Dependency Injection	Fails with 401 if token is invalid or missing.	Identity confirmed.
5	FastAPI (Validation)	Re-validates the uploaded image file	Server-side validation function	Fails with 422 if file is corrupt or wrong type	Valid image file is passed to service.
6	FastAPI (Service)	Preprocesses image and runs inference.	MobileNetV3 model invocation	Handles model timeout/failure exceptions.	Quality grade and confidence generated.
7	FastAPI (Data Access)	Saves image reference and classification result	Parameterized INSERT SQL query via ORM	Verifies foreign key constraints (User ID)	Database transaction committed.
8	PostgreSQL	Persists the records	Database engine execution	Enforces schema constraints designed in IT 108.	Records safely stored.
9	FastAPI (Response)	Constructs the JSON response body.	Returns HTTP 201 Created with JSON result	Strips internal server data from response	Response sent back to client.
10	Flutter Web	Parses JSON response and updates state.	Riverpod state update	Handles network error or success payload.	UI displays the final classification grade to the user.

Tracing Guidance

A strong trace normally includes: (1) user action in Flutter, (2) client-side validation, (3) request creation with method/path/headers/body, (4) route matching, (5) authentication/authorization, (6) server-side validation, (7) service/business-rule execution, (8) PostgreSQL query or transaction, (9) response generation with status + JSON, and (10) Flutter state update and user feedback.

7. Request–Response Trace Check

- ☐ The trace uses one real use case from Activity 5.
- ☐ The starting screen exists in the Activity 6 UI flow.

- ☐ The endpoint has a meaningful HTTP method and resource-oriented path. ☐ Server-side validation is shown even when Flutter also validates the form. ☐ The database step references existing IT 108 data domains rather than recreating the schema. ☐ Both success and at least one failure path are identified.

8. Part C – Construct the API Flow Diagram

Create one clean, readable API Flow Diagram showing how a real request travels from Flutter Web through the REST API backend to PostgreSQL, and how the response returns to the interface. The diagram must describe your own project—not a generic sample.

No.	Required Element	What Must Be Shown
1	Flutter Web client	Identify the actual screen/widget or user action that starts the request.
2	Request	Show HTTP method, route/path, HTTPS, headers/token when required, and JSON/body or query parameters.
3	Route / endpoint	Show where the request enters the server-side application.
4	Controller / handler	Show request parsing, response coordination, and delegation to the service layer.
5	Validation & security	Show server-side validation, authentication, authorization, and protected operations where applicable.
6	Service / business logic	Show the rule, calculation, approval, recommendation, or workflow operation performed.
7	Data access	Show repository/query/data-access responsibility without embedding raw database design in the diagram.
8	PostgreSQL	Reference the relevant IT 108 data domain/table/entity names.
9	External service	Include only if the selected workflow actually uses one.
10	Response	Show returned status code, JSON/result, and Flutter state/UI feedback.
11	Error path	Show at least one structured failure path such as invalid input, unauthorized access, not found, conflict, or server error.
12	Legend	Use consistent shapes, arrows, and labels.

9. Reference Example – Do Not Copy as Your Final Output

The figure below demonstrates the expected level of clarity for a Flutter Web → REST API → PostgreSQL request lifecycle. Replace all labels with the actual endpoint, business logic, security checks, data domains, and response behavior of your own system.

ence: Request-Response Flow in a Flutter Web → REST API → PostgreSQL Sy

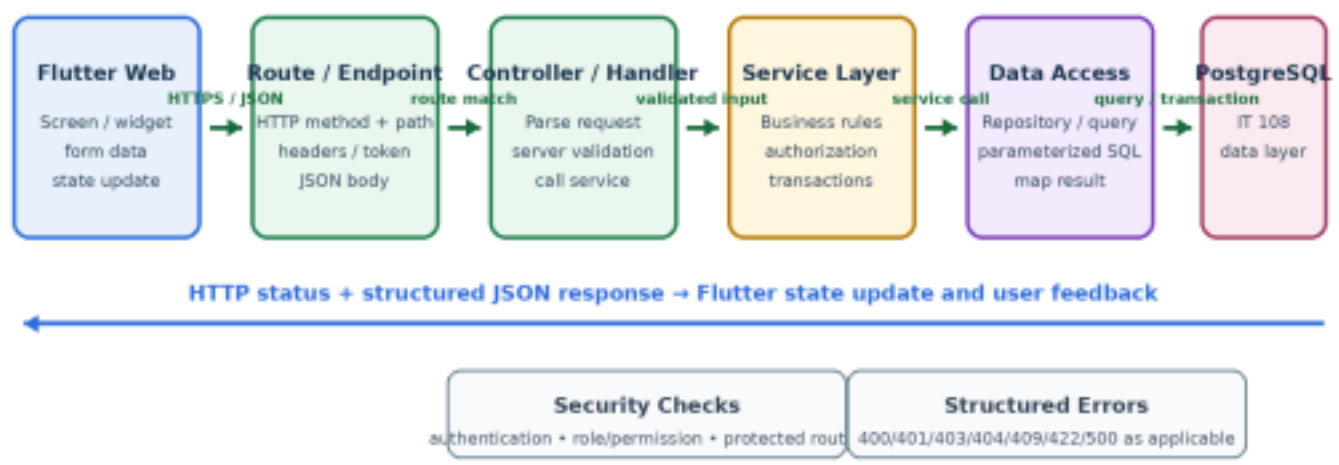


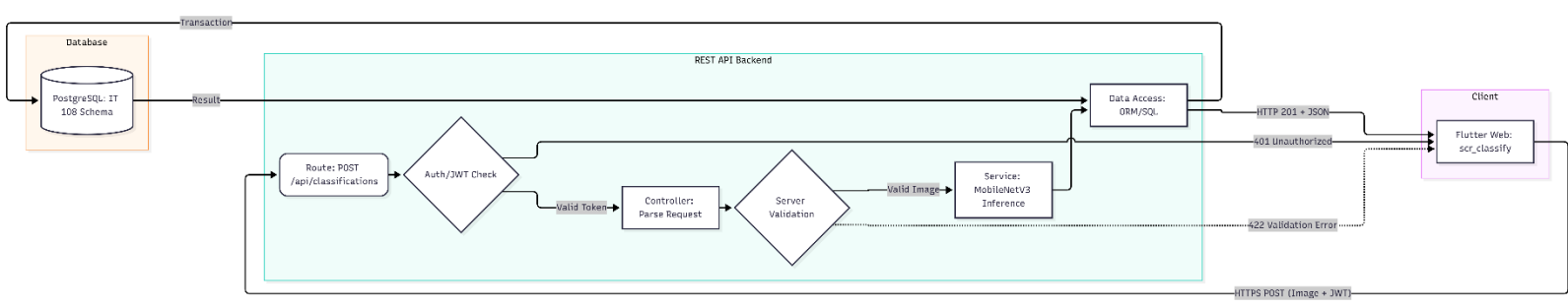
Figure 1. Conceptual request–response path for a Flutter Web client using a REST API and PostgreSQL.

Minimum Flow Rule

Your final diagram must show at least one complete path: Flutter screen/user action → HTTP request → endpoint/route → handler/controller → server validation/security → service/business logic → PostgreSQL or external service → HTTP status + JSON response → Flutter state/UI result. Every arrow on this path must have a meaningful label.

10. Recommended API Flow Labels

Area	Possible Label	What It Represents
Client	Flutter Web screen / widget	User input, event, client-side validation, state update
Transport	HTTPS + REST + JSON	Method, path, headers, body/query parameters
Routing	Endpoint / route	Matches request to server-side handler
Handler	Controller / request handler	Parses request and coordinates response
Service	Service / use-case layer	Business rules, permissions, workflow logic
Data Access	Repository / query layer	Reads/writes PostgreSQL through safe parameterized access or ORM
Security	Auth middleware / guards	Identity, token verification, role/permission checks
Response	HTTP status + JSON	Success result or structured error for Flutter



11. Part D – Semester-Project Endpoint Inventory

List the initial endpoints needed to support the approved core features. Use resource-oriented paths and select methods based on the intended action. Do not create endpoints for features outside the approved scope.

ID	Requirement / Use Case	HTTP Method	Route / Path	Authorized Role(s)	Handler / Service	PostgreSQL Data Domain
EP-01	FR-01: Register account	POST	/api/auth/register	Public	Auth Controller / Create User logic	Users
EP-02	FR-03: Login/authentication	POST	/api/auth/login	Public	Auth Controller / Verify Credentials	Users
EP-03	FR-09: Dashboard data	GET	/api/users/me/dashboard	Farmer, Seller	User Controller / Fetch Summary	Users, History
EP-04	FR-04, 05, 06: Classify Guso	POST	/api/classifications	Farmer, Seller	Classification Controller / MobileNetV3	Guso_Images, Classification_Results
EP-05	FR-08: View History	GET	/api/classifications/history	Farmer, Seller	History Controller / Fetch Records	Classification_Results, History
EP-06						
EP-07						
EP-08						

12. REST Method and Route Selection Guide

Method	Typical Purpose	Example	Design Note
GET	Retrieve a resource or collection	GET /api/reports; GET /api/reports/{id}	Should not create side effects.
POST	Create a resource or trigger a non idempotent operation	POST /api/reports	Usually returns 201 Created when a resource is created.
PUT	Replace a complete resource when the API is designed that way	PUT /api/users/{id}	Use consistently; often idempotent.
PATCH	Partially update a resource	PATCH /api/reports/{id}	Use for partial changes such as status or selected fields.
DELETE	Remove a resource when deletion is allowed	DELETE /api/reports/{id}	Often returns 204 No Content after successful deletion.

Route Naming Reminder

Prefer nouns/resources in paths, keep naming consistent, avoid exposing raw database table operations directly, and make endpoint names understandable from the system's domain. Exact conventions may vary with the selected backend framework, but the contract must remain clear and consistent.

13. Part E – Initial API Contract

Document at least FOUR core endpoints from the inventory. Two of the four must be written in full detail using the contract template below; the other two may be summarized in the endpoint inventory if time is limited during the laboratory period.

Contract ID / Feature	EP-02 / User Login
HTTP Method + Route	POST /api/auth/login
Purpose / Related Use Case	Verify credentials and issue JWT token (UC/US-02).
Authorized Role(s)	Public
Required Headers	Content-Type: application/json
Path / Query Parameters	None
Request JSON Body	{"contact_number": "string", "password": "string"}
Server-Side Validation	Check fields are present; verify password hash against database.
Success Response	HTTP 200 OK {"token": "eyJhb...", "user_id": 1}
Error Responses	HTTP 401 Unauthorized {"error": {"code": "AUTH_FAILED", "message": "Invalid credentials"}}
PostgreSQL Data Domain	Users

14. Contract Quality Rules

- The contract must be understandable without looking at the implementation code.
- The route and HTTP method must agree with the feature and request-lifecycle trace.
- Client-side validation from Activity 6 must be repeated or strengthened on the server when the rule protects data integrity or access.
- Do not return passwords, tokens, secrets, internal stack traces, raw SQL, or unnecessary private fields in responses.
- Use consistent field names and JSON types across requests and responses.
- Document expected failure cases instead of treating every failure as a generic 500 error.

15. HTTP Status Codes and Structured Responses

Status	Typical Meaning in the API Contract
200 OK	Successful retrieval/update when a response body is returned.
201 Created	New resource successfully created.
204 No Content	Successful operation with no response body, commonly deletion or update.
400 Bad Request	Malformed request or invalid request syntax/structure.
401 Unauthorized	Authentication is missing or invalid.
403 Forbidden	User is authenticated but lacks permission.
404 Not Found	Requested resource does not exist or is not available to the requester.
409 Conflict	Request conflicts with current resource state, such as duplicate or invalid transition.
422 Unprocessable Content	Request is structurally valid but fails semantic/field validation when this convention is used.
500 Internal Server Error	Unexpected server failure; return a safe message and log details server-side.

16. JSON Request / Response and Error Design

Use valid JSON with consistent field names and types. Your actual contract may use a different structure, but success and error responses must be predictable for the Flutter client.

Example Success	Example Structured Error

HTTP 201 <pre>{ "id": 125, "status": "submitted", "message": "Report created" }</pre>	HTTP 422 <pre>{ "error": { "code": "VALIDATION_ERROR", "message": "Some fields are invalid", "details": { "location": "Required" } } }</pre>
--	---

Status-Code Reminder

The table provides common REST conventions, not a rule that every project must use every status. Select the codes that accurately describe your API behavior, document them in the contract, and apply them consistently.

17. Part F – Server-Side Validation and Error Matrix

For one form or write operation from Activity 6, identify the rules that must be enforced on the server even if Flutter already performs client-side validation.

Field / Rule	Client-Side Check	Server-Side Check	Business / Security Reason	Error Status	Safe Error Message / Code
Image Upload	Check extension (.jpg, .png) and size (<5MB)	Validate true MIME type, scan for corruption, enforce strict size limit	Malicious files could compromise the server or crash the ML model	422	Invalid image format. Please upload a valid JPG or PNG.
Authentication Token	Check if token exists in local state before routing	Cryptographically verify the JWT signature and expiration.	Client state can be manipulated; the server must strictly authorize access.	401	Session expired or unauthorized. Please log in again.
Record Retrieval (History)	Only requests history for the logged-in user.	Extract user ID from JWT; append WHERE user_id = ? to SQL query.	Prevent users from manually altering request parameters to view competitors' data.	403	You do not have permission to access these records.

18. Part G – Conceptual API-to-PostgreSQL Mapping

Connect API operations to the existing PostgreSQL data layer from IT 108 at a conceptual level. Do not recreate the ERD, normalization, keys, constraints, or physical schema here.

Endpoint	Service / Business Operation	Data Action	IT 108 Data Domain / Entity	Returned Result
POST /api/auth/register	Hash password, create user	INSERT	Users	User ID
POST /api/auth/login	Verify hash against stored user	SELECT	Users	JWT Token
POST /api/classifications	Process image, execute MobileNetV3	INSERT	Guso_Images, Classification_Results	Classification Grade & Confidence
GET /api/classifications/history	Fetch records owned by user	SELECT	History, Classification_Results	Array of record objects

IT 108 Boundary

Reference the approved IT 108 table/entity or data-domain names only to show where the API reads or writes. Do not recreate or redesign the database schema in this Activity 7 manual.

19. Laboratory Procedure

Phase 1 – Select a critical workflow (10 minutes)

Choose one high-value use case that begins in an Activity 6 Flutter screen and requires the REST API and

PostgreSQL. Phase 2 – Trace the request lifecycle (20 minutes)

Complete Part B from user action to Flutter request, route, validation/security, service logic, data access, response, and UI feedback.

Phase 3 – Draft the API Flow Diagram (25 minutes)

Draw the complete system-specific request/response path. Label methods, paths, data, validation, security checks, results, and errors.

Phase 4 – Plan endpoints (20 minutes)

Build the endpoint inventory for the approved core features. Review method selection, path naming, roles, services, and data domains.

Phase 5 – Write the initial API contract (30 minutes)

Document at least four core endpoints; fully detail two using the contract template.

Phase 6 – Validate responses and errors (10 minutes)

Check JSON bodies, HTTP status codes, headers, field validation, authorization, and structured

errors. **Phase 7 – Review API-to-PostgreSQL mapping (10 minutes)**

Confirm that the API references the current IT 108 data domains without duplicating database design. **Phase 8 – Final review and submission preparation (15 minutes)**

Use the checklist, update the diagram and contract, export required files, and commit documentation to

GitHub. **20. Final Quality Checklist**

☐ The API Flow Diagram uses the official system title and a real critical workflow.	☐ JSON request/response fields and data types are consistent.
☐ The starting Flutter screen/user action exists in Activity 6.	☐ Success and failure status codes are documented.
☐ At least one complete request-and-response path is visible and labeled.	☐ Errors are structured and do not expose internal or sensitive details.
☐ HTTP methods and route paths are meaningful and consistent.	☐ PostgreSQL references agree with the IT 108 data layer and do not duplicate schema design.
☐ Controllers/handlers and service/business logic are not treated as the same responsibility without explanation.	☐ Endpoint inventory is traceable to approved requirements/use cases.
☐ Server-side validation is included even when Flutter validates the same input.	☐ At least four core endpoints are documented and two are fully detailed.
☐ Authentication and authorization are shown for protected endpoints.	☐ Every group member can explain the selected request lifecycle and API contract.

21. Submission Requirements

- PDF or PNG of the final API Flow Diagram in readable landscape orientation.
- Editable diagram source file (.drawio, .pptx, .fig, or native file of the chosen tool).
- Completed request-lifecycle tracing worksheet.
- Endpoint inventory and Initial API Contract with at least four core endpoints; at least two endpoints must contain full request/response, validation, status-code, and error details.
- Completed server-side validation/error matrix and conceptual API-to-PostgreSQL mapping.
- Commit the files to the project GitHub repository under a folder such as docs/api/ or docs/backend/. • Submit the repository link and required files through the class submission platform using the naming convention: IT110_LA7_Group##_SystemName.

Academic Integrity and Competency Validation

The API design must represent the team's own approved system and must be consistent with its requirements, use case diagram, Flutter UI flow, and IT 108 data layer. AI tools may be used to clarify concepts or improve documentation, but every endpoint, validation rule, role restriction, request/response field, and flow decision must be reviewed and understood by the team. The instructor may ask any member to trace a request or explain an endpoint without assistance.

22. Scoring Rubric – 80 Points

		Points
API flow correctness and completeness	The diagram accurately traces Flutter → REST API → PostgreSQL → response, including routes, handlers, validation/security, service logic, data access, and error/success paths.	15
Endpoint planning and REST design	Core endpoints are traceable to approved features; methods and resource-oriented paths are appropriate, consistent, and limited to the project scope.	12
Initial API contract completeness	At least four core endpoints are documented and the required detailed contracts clearly define purpose, roles, parameters, headers, request/response bodies, validation, status codes, and errors.	15
HTTP semantics and JSON design	Methods, status codes, request/response fields, JSON types, headers, and result structures are technically coherent and consistently documented.	10
Server-side validation, security, and errors	Validation, authentication, authorization, protected operations, safe error messages, and structured failure responses are clearly planned.	10
Conceptual API-to-PostgreSQL mapping	Endpoints and service operations map correctly to existing IT 108 data domains without duplicating database design.	8
Traceability and implementation readiness	The API plan aligns with Activity 5 requirements/use cases and Activity 6 Flutter screens and can guide subsequent backend implementation.	5
Documentation quality and technical clarity	Tables, labels, terminology, diagrams, and file organization are readable, consistent, and professionally prepared.	5

23. Post-Laboratory Reflection

Which endpoint in your system is the most security-sensitive, and what authentication/authorization checks must occur before its business logic runs?

The POST /api/auth/login and subsequent protected endpoints (like /api/classifications) are highly sensitive. Before any classification logic runs, FastAPI must explicitly decode the JWT token, verify its cryptographic signature, ensure it hasn't expired, and confirm the user ID maps to an authorized Farmer or Seller role in the database.

Which validation rule must be enforced on the server even if Flutter already checks it, and why?

The image file validation (checking MIME type and file integrity) must be strictly enforced on the server. Even if the Flutter Web client blocks bad files, a malicious user can easily bypass the browser and send an API request directly (e.g., using Postman). If the server doesn't re-validate, a corrupt or malicious payload could crash the FastAPI backend or the MobileNetV3 model.

Which API response will be the most difficult for the Flutter client to handle correctly, and what UI state should be shown?

Where is the clearest boundary between the controller/handler and service/business-logic responsibilities in your selected workflow?

The boundary is immediately after the endpoint receives and validates the image file. The Controller's job stops at extracting the file and verifying the JWT token. It then passes the raw image data to a separate Service function,

which is solely responsible for resizing the image, running the MobileNetV3 model, and returning the raw grade back to the Controller.

What API contract detail must your team finalize before beginning the next server-side implementation activity?

We must finalize the exact JSON field names (e.g., `contact_number` vs `phone_number`) and data types (string vs integer for IDs) between the Flutter frontend team and the FastAPI backend team to ensure there are no parsing errors when we begin integration testing.